AllHands: Ask Me Anything on Large-scale Verbatim Feedback via Large Language Models

Chaoyun Zhang, Microsoft, China
Zicheng Ma*, ZJU-UIUC Institute, China
Yuhao Wu*, National University of Singapore, Singapore
Shilin He, Microsoft, China
Si Qin, Minghua Ma, Microsoft, China
Xiaoting Qin, Yu Kang, Microsoft, China
Yuyi Liang, Xiaoyu Gou, Microsoft, China
Yajie Xue, Microsoft, China
Qingwei Lin, Microsoft, China
Saravan Rajmohan, Microsoft, USA
Dongmei Zhang, Microsoft, China
Qi Zhang, Microsoft, China

arXiv:2403.15157v2 [cs.SE] 3 Apr 2024

ABSTRACT

Verbatim feedback constitutes a valuable repository of user experiences, opinions, and requirements essential for software development. Effectively and efficiently extracting valuable insights from such data poses a challenging task. This paper introduces AllHands, an innovative analytic framework designed for large-scale feedback analysis through a natural language interface, leveraging large language models (LLMs). AllHands adheres to a conventional feedback analytic workflow, initially conducting classification and topic modeling on the feedback to convert them into a structurally augmented format, incorporating LLMs to enhance accuracy, robustness, generalization, and user-friendliness. Subsequently, an LLM agent is employed to interpret users' diverse questions in natural language on feedback, translating them into Python code for execution, and delivering comprehensive multi-modal responses, including text, code, tables, and images.

We evaluate AllHands across three diverse feedback datasets. The experiments demonstrate that AllHands achieves superior efficacy at all stages of analysis, including classification and topic modeling, eventually providing users with an "ask me anything" experience with comprehensive, correct and human-readable response. To the best of our knowledge, AllHands stands as the first comprehensive feedback analysis framework that supports diverse and customized requirements for insight extraction through a natural language interface.

1 INTRODUCTION

Verbatim feedback frequently encapsulates users' experiences, opinions, requirements, and specific inquiries about software, applications, or other tangible and virtual products [1, 2]. It serves as a crucial conduit for developers to comprehend the "voice of users", offering insights into both favorable and unfavorable aspects of a product, identifying shortcomings, and elucidating desired new features [3]. The analysis of such feedback data plays a pivotal role in streamlining the product release and upgrade processes for developers. This entails addressing existing issues, introducing new features, and resolving user confusion, ultimately contributing to the overall enhancement of the product [4]. This makes the practice especially vital in the realm of software requirement engineering, where developers routinely gather feedback from diverse channels to extract meaningful insights [5–7].

Effectively and efficiently deriving meaningful insights from users' feedback poses a nontrivial challenge, particularly in the context of globally released, popular software with a substantial volume of feedback. This challenge is exacerbated by users posting reviews across diverse platforms, utilizing different languages, and generating thousands of reviews daily [8]. The manual examination of each piece of feedback becomes an infeasible task. Furthermore, developers employ varied dimensions and tools for feedback analysis, adapting their approaches for different software at various stages of development. For instance, in the early stages of development, developers aim to comprehend the anticipated functions and requirements of the software [9]. As the software matures, the focus shifts to identifying existing issues [10], understanding user experiences [11], comparing with historical versions [12], and discerning the most critical aspects that warrant improvement [13]. These diverse analysis requirements, coupled with the large-scale influx of feedback from heterogeneous sources, present substantial challenges to the process of feedback analysis.

Automating feedback analysis commonly involves initial steps such as classifying feedback into predefined dimensions [14] or conducting topic modeling to decompose each feedback into distinct topic dimensions [15]. This transformation converts non-structural textual feedback into a structural format, enriched with various features conducive to analytical processes. Consequently, diverse tools can be developed to operate on the structural feedback, catering to a range of analysis requirements [16]. Traditionally, the classification and topic modeling processes have heavily relied on various machine learning or natural language processing (NLP) models, such as BERT [17] and Latent Dirichlet Allocation (LDA) [15]. Subsequent analytics are often ad-hoc and customized to specific requirements. However, we acknowledge several limitations inherent in existing

*This work was completed during their internship at Microsoft.

solutions. Specifically, for the classification task, substantial human-labeled data and effort are typically required for model training in specific domains, making generalization challenging. In the context of topic modeling, an extractive approach is commonly employed [18], where each topic is represented as a combination of key words extracted from the documents. This approach may struggle to handle challenges such as polysemy and multilingual scenarios, and it lacks human readability and coherence in topic representation. Furthermore, developers need to create ad-hoc analytic tools to extract meaningful insights from feedback, tailored to their specific requirements. Given the diverse set of requirements, this demands significant human effort and is particularly unfriendly to users lacking coding proficiency.

To overcome the aforementioned limitations, this paper introduces a comprehensive analytic framework for large-scale verbatim feedback named AllHands, harnessing the capabilities of large language models (LLMs). AllHands serves as an all-encompassing solution for feedback classification, abstractive topic modeling, and the ultimate extraction of insights. It offers a user interface where users can pose analytic questions in natural language and receive responses in the form of text, code, tables, and even images. This framework accommodates diverse requirements for insight extraction from feedback in real-world scenarios, providing answers with comprehensive multi-modal outputs and enabling true "ask me anything" capabilities in large-scale verbatim feedback.

AllHands adheres to a workflow akin to traditional feedback analytic approaches, involving the initial structuralization of textual feedback through classification and topic modeling, followed by subsequent insight extraction. However, it enhances each stage by integrating LLMs, resulting in more accurate, robust, generalized, and user-friendly outcomes and experiences throughout the analytic process. In the classification phase, AllHands employs LLMs with in-context learning (ICL) [19] to precisely categorize feedback into any predefined dimension using limited few-shot demonstrations, thereby eliminating the need for model fine-tuning. Additionally, AllHands utilizes LLMs for abstractive topic modeling, summarizing each feedback into human-readable topics that align with user-defined criteria. This ensures improved topic relevancy and coherence. Crucially, AllHands integrates an LLM-based agent to translate users' natural language questions about feedback into Python code. This agent operates on the structural feedback data, delivering answers to users interactively in a multi-modal format. Capable of addressing a wide range of common feedback-related questions, the framework is extensible with self-defined plugins for more complex analyses. Consequently, AllHands emerges as a fully automated and user-friendly feedback analytic framework.

We conduct a systematic evaluation of AllHands's performance using three diverse feedback datasets, demonstrating its superior efficacy across all stages. Overall, this paper contributes in the following ways:

- We introduce AllHands, a comprehensive feedback analytic framework that serves as a one-stop solution for classification, topic modeling, and question answering (QA) using LLMs, enabling a "ask me anything" approach for large-scale feedback analysis.

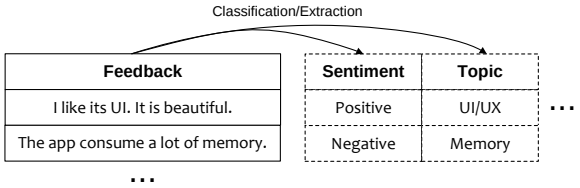

Figure 1: Overview of feedback classification and topic extraction.

- The application of LLMs with ICL in the AllHands classification phase achieves superior accuracy across all feedback datasets without the need for model fine-tuning.
- AllHands utilizes LLMs for abstractive topic modeling, delivering customized and human-readable topic representations that exhibit enhanced relevancy and coherence.
- The LLM-based agent embedded in AllHands facilitates flexible and extensible feedback QA through a natural language interface, providing comprehensive outputs through a multi-modal approach.

To the best of our knowledge, AllHands stands as the first feedback analysis framework designed to accommodate diverse requirements for insight extraction through a natural language interface, thereby revolutionizing new avenues for future research.

2 BACKGROUND

This section presents an overview of feedback classification and unsupervised topic modeling, as well as the background for extracting insights from feedback data. These elements collectively form the foundational framework of AllHands.

2.1 Feedback Classification and Topic Extraction

Feedback comprises textual data that conveys the opinions or experiences of users regarding specific products [20], applications [21], or other entities [22]. This data proves invaluable to developers as it provides insights and guidance for enhancing their products or applications. To analyze batches of feedback data effectively and efficiently, a prevalent approach involves extracting pertinent topics or features from each piece of feedback [23, 24]. This process transforms the data into a structured format, facilitating subsequent analysis. Such transformation can be accomplished through text classification when labeled training data is accessible [25], or alternatively through unsupervised approaches for topic extraction [26], as shown in Fig. 1.

Feedback classification involves categorizing each textual feedback into specific dimensions, such as informativeness, sentiment, and topics of interest. This task is typically accomplished through supervised machine learning models trained on labeled datasets [14, 27]. Conversely, topic extraction aims to identify and extract implicit themes or topics from textual data, and label each topic with textual representation, facilitating the summarization and comprehension of large volumes of text. This process is useful for automating information retrieval, organization, and analysis, with the objective of determining key themes in a text rather than merely identifying keywords. Unsupervised techniques, including

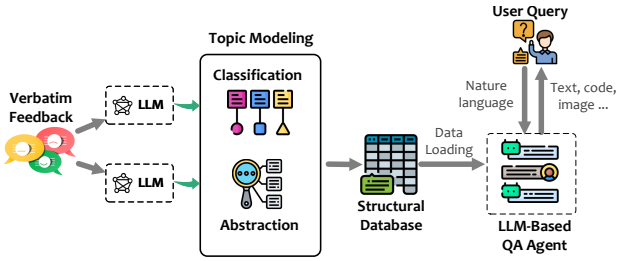

Figure 2: The overall architecture of AllHands.

clustering [28] and LDA [15], are often employed for topic extraction. Both classification and topic extraction contribute additional feature dimensions to feedback, enhancing insights and analytical capabilities.

2.2 Insight Extraction from Feedback

Numerous analysis paradigms leverage classification and topic extraction as foundational steps to derive insights from feedback data, thereby aiding in future improvements. These analyses span diverse dimensions, encompassing tasks such as emerging issue identification [29, 30], correlation analysis [31, 32], causal analysis [33, 34], and evolution analysis [35, 36], among others. These analysis can offer invaluable insights to product developers. The classified and extracted topics or features serve as crucial identifiers, grouping feedback data and facilitating the extraction of valuable insights. While various tools have been developed to support specific analysis objectives, there remains a gap in the availability of a flexible and unified framework that can accommodate a wide array of analyses.

2.3 System Objective

AllHands effectively bridges this gap by harnessing the capabilities of LLMs. In essence, AllHands is designed to accept user queries in natural language regarding feedback data and provide answers in diverse formats such as text, code, and images. To accomplish this, AllHands divides the overarching task into two main components: *(i)* topic classification/extraction and *(ii)* feedback QA. In the first stage, each feedback is enriched with additional topics and features through the process of topic classification/extraction. Then, the feedback QA stage utilizes an LLM-based agent to translate user queries into Python code, delivering execution results and summarizations as needed. The QA agent is adept at addressing a broad spectrum of questions related to the feedback data, as outlined in Sec. 2.2, thereby enabling a genuine "ask me anything" capability to provide insights into feedback analysis.

3 THE DESIGN OF ALLHANDS

We overview the overall architecture of AllHands in Sec. 3.1, and detail each of its component in the following subsection.

3.1 AllHands in a Nutshell

Fig. 2 presents the overarching architecture of the AllHands framework. The anticipated input for AllHands comprises a substantial batch of unstructured verbatim textual feedback. The primary objective of AllHands is to facilitate the extraction of a diverse array of insights from this feedback through natural language queries.

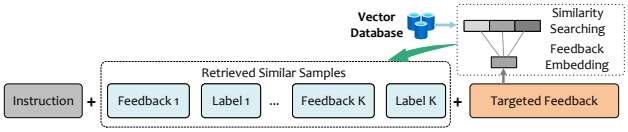

Figure 3: The structure of a prompt employed in the feedback classification.

Upon receiving the unstructured feedback data, the initial step involves the extraction of topics or features, such as sentiment and informativeness, for each piece of feedback. This transformation is aimed at converting the unstructured data into a structured format, thereby facilitating subsequent Question Answering (QA) processes. Achieving this entails either classification into various dimensions, if labeled training data is available, or the extraction of new topics related to the feedback through an unsupervised approach. This process, referred to as topic modeling, can be executed by leveraging LLMs without the need for fine-tuning. Further details are provided in Sec. 3.2 and 3.3.

The aforementioned process involves augmenting each feedback instance with additional features, effectively translating the original unstructured feedback into a structured database. This structured database facilitates more straightforward analysis and querying through programming languages like Python. In the subsequent step, AllHands integrates a LLM-based question answering agent, designed to interpret ad-hoc user queries in natural language, translate them into executable code, execute the code, and subsequently return the results to users. The returned results can manifest in various forms, including text, code, and even images generated by drawing libraries. This diverse output capability ensures comprehensive answers to users, achieving the ultimate objective of a "ask me anything" capability on the feedback data to deliver insights. Further elaboration on this agent is provided in Sec. 3.4.

3.2 Feedback Classification

Pretrained LLMs, such as the GPT series [37], are typically trained on extensive volumes of general information sourced from publicly available domains or the Internet. They have demonstrated effectiveness in classifying textual data across diverse domains, often without the need for fine-tuning, especially when the data is not highly domain-specific. This characteristic makes them particularly well-suited for feedback classification, given the inherently diverse and generative nature of the data context. AllHands leverages the few-shot learning capability of LLMs [38], harnessing the ability to inject necessary context into the model and provide examples for demonstration. This approach, also known as the in-context learning (ICL) [19] enhances the model's classification effectiveness in handling the nuances of feedback data.

To effectively harness labeled data, AllHands initially employs the sentence transformer [39] to vectorize all labeled data, storing them in a vector database [40]. During the classification process, the input feedback is embedded using the same embedding model. Subsequently, the top-K similar samples are retrieved using the cosine similarity [41] metric. These retrieved samples are then utilized to construct the prompt sequence for the LLM, with the following detailed components.

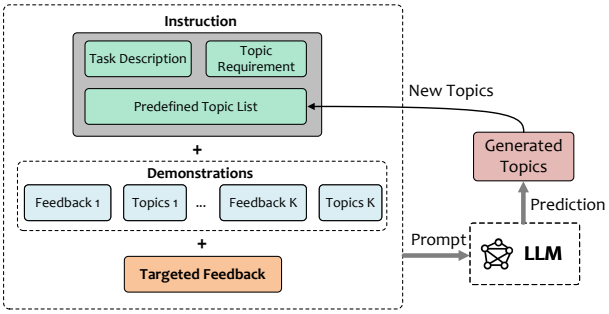

Figure 4: The progressive ICL process for abstrative topic modeling.

In In-Context Learning (ICL), the prompt typically comprises three components, namely *(i)* An instruction providing background information, guidelines, and the objective of the classification. *(ii)* The retrieved top-K similar samples, including the feedback and their ground truth labels, serving as demonstrations. *(iii)* The targeted feedback to be classified. An illustrative example of the prompt structure is provided in Fig. 3. LLM can then generate the predicted category of the given feedback based on the constructed prompt.

In contrast to smaller language models (*e.g.*, [42, 43]), LLMs can execute classification without the need for retraining and possess the ability to generalize across different domains or dimensions. This capability stems from their capacity to leverage knowledge acquired during extensive pretraining, obviating the necessity for large amounts of labeled data. Additionally, LLMs can provide more accurate classifications, as demonstrated in Sec. 4.2. The LLM-based feedback classifier serves to extend the feedback data into designated dimensions, a crucial step in subsequent analyses, and represents a significant component of AllHands.

3.3 Abstractive Topic Modeling

Traditional topic modeling methods applied to user reviews, such as LDA and its variants, often adopt an "extractive" approach. This involves assuming that each review or document is composed of a mixture of topics, with each topic represented as a combination of key words extracted from the documents. These models adhere to a bag-of-words assumption [44], which may overlook order and contextual information. Furthermore, they may struggle to handle challenges like polysemy and multilingual scenarios. While various solutions employing neural networks (*e.g.*, [45–47]) have been proposed to partially address these challenges, many of these models still represent each topic as a word distribution. This representation may lack human readability and coherence [18, 48].

To address these limitations, AllHands employs LLMs to summarize each review into one or multiple phrases, facilitating abstractive topic modeling. Unlike traditional methods that rely on extracting key words from raw text, these phrases serve as high-level summarizations tailored to the context of the review [49], ensuring they are more human-readable and conducive to analysis. Furthermore, AllHands can guide LLMs to distill specific directional aspects of topics of interest through instructions and demonstrations, and it has the capability to identify new topics over time, thus overcoming challenges associated with extractive topic modeling.

3.3.1 In-context Abstractive Topic Modeling. To this end, AllHands employs abstractive topic modeling through ICL, akin to the technique described in Sec. 3.2. However, in this context, the prompt is updated progressively with new topics found. The overall pipeline for this process is illustrated in Fig. 4. In contrast to the instructions provided in the prompt for classification, the instructed prompt for abstractive topic modeling offers more specific information, including:

- **Task description:** Providing background information on the data and outlining the overarching objective of abstractive topic modeling.
- **Topic requirement:** Detailing specific requirements for the desired topics to be extracted, such as directions, levels, and other customizable criteria to align with practical requirements.
- **Predefined topic list:** Supplying examples of topics that align with the specified requirements, serving as candidates for a cold start. Additionally, new topics can be generated in addition to this predefined list.

The instructed prompt is complemented by several demonstrated typical examples falling into the predefined topics, serving as contextual information. Additionally, the targeted feedback is fed to LLMs for topic summarization. LLMs predict one or multiple topics for each feedback sequentially, for instance, based on the time of posting. As new topics are generated, they are added to the predefined topic list for subsequent feedback, ensuring that emerging topics can be detected in addition to the predefined ones. This marks the completion of the first round of unsupervised abstractive topic modeling, generating customized, human-readable, and diverse topics for each feedback.

3.3.2 Human-in-the-Loop Refinement. While LLMs excel in summarizing textual data [50], their generated topics may lack controllability and, in some cases, may not fully satisfy expectations. This is particularly evident in the first round when limited information is provided to LLMs. To address this limitation, we propose leveraging the output from the first round and applying a human-in-the-loop approach to enhance the quality of abstractive topic modeling. The process is illustrated in Fig. 5.

After obtaining the topic modeling results in the first round, we compile the unique topic paraphrases generated. A reviewer is tasked with selecting the topics of interest and removing any that do not align with the customized criteria. These could include long-tailed topics, those out of scope, or irrelevant ones. Subsequently, we employ hierarchical agglomerative clustering [51] on the remaining topics, utilizing their embeddings vectorized by the sentence transformer [39]. Following this, LLMs are employed to summarize each cluster into a high-level phrase for new representation. This process yields a new set of topics for consideration, and the reviewer is responsible for further filtering. Once completed, the predefined topic list and demonstrated typical examples are updated with the new topic set.

Additionally, we utilize the results from the first round to construct a vector database and retrieve an additional N examples

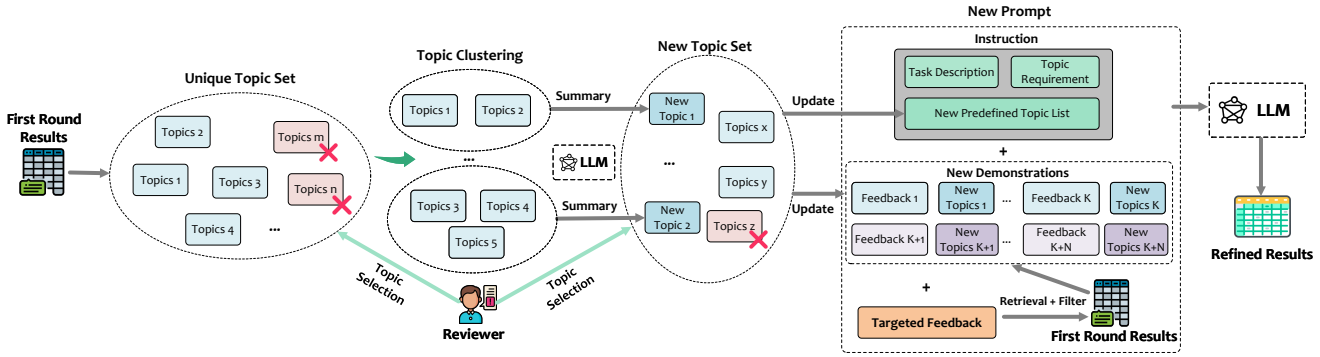

Figure 5: The process of the human-in-the-loop topic refinement employed in AllHands.

(denoted as purple feedback in Fig. 5) based on text similarity using their embeddings for targeted feedback. It is important to note that we filter out those topics extracted in the first round with low BARTScore [52] compared to the original feedback in the vector database, as these topics may be considered low quality and may not effectively summarize the feedback. These additional N examples are appended to the end of the fixed few-shot samples to reinforce context, providing more demonstrations that ultimately enhance the quality of topics. Subsequently, the new prompts are submitted to the LLM to conduct the second round of abstractive topic modeling, resulting in new outcomes that are more aligned with the specified requirements.

The human-in-the-loop refinement in the second round effectively infuses human knowledge into the modeling process, while minimizing human effort. The reviewer is required to make judgments on a limited set of topics rather than each feedback individually. This approach retains the most useful topics of interest while removing dissatisfying ones, resulting in outcomes that are more aligned with the specified requirements. The clustering-and-summarize approach further aggregates similar topics. It's important to note that this process can be iterated multiple times to further enhance the topic modeling.

In summary, abstractive topic modeling, coupled with topic classification, introduces new feature dimensions to verbatim feedback, transforming non-structural textual data into structured data. This transformation significantly facilitates analysis and insights extraction, bridging the gap between feedback data and existing analysis and query tools that operate more effectively on structured data. Moving forward, we introduce how to leverage LLMs to extend this bridge to users' questions in natural language, enabling a "ask me anything" capability on the feedback.

3.4 "Ask Me Anything" with an LLM-based QA Agents

Finally, we develop a Question Answering (QA) agent [53] to facilitate "Ask Me Anything" functionality on the structured feedback data acquired in previous steps. This entails three primary tasks:

- Converting user queries in natural language into executable code.
- Executing the code to generate results.
- Summarizing the execution results as necessary and providing responses to the user.

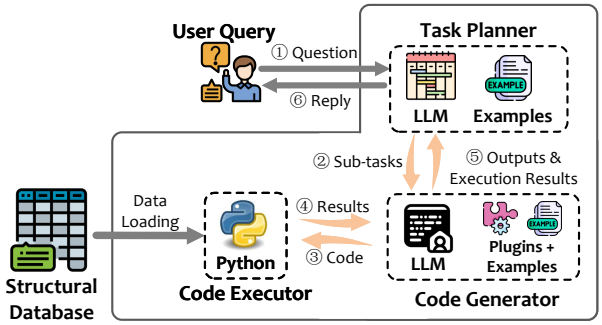

Figure 6: The overall architecture of AllHands.

To the end, we design a code-first LLM agent framework to converts user request into executable code, supports rich data structures, flexible plugin usage, and leverages LLM coding capabilities for complex logic, which is particularly well-suited for QA tasks on feedback data. The overall architecture is depicted in Fig. 6.

Overall, this agent is comprising a task planner, a code generator, and a code executor. The task planner, driven by an LLM, plays a pivotal role in converting user queries into multiple substeps and dispatches Code Generator (CG) queries to the LLM-based Code Generator. The code generator, in turn, generates executable code or invokes necessary plugins based on the CG query and sends the code to the code executor for execution. The code executor returns results to the code generator for potential code revisions, if required. Upon receiving the final outcome, the task planner summarizes the results and can respond to users in diverse formats such as code, tables, images, or natural language. We provide detailed description of each component next.

3.4.1 Planner. The planner plays a crucial role in the QA process. It accepts users' queries in natural language, decomposes these requests into several sub-tasks [54–56], and orchestrates and organizes the capabilities within the QA agent. Once the request is fulfilled, the planner replies back to the users in various forms. Essentially, it serves as the entry point and controller, managing the entire QA framework. The planner engages in bidirectional communication with each component, sending queries and receiving responses, and subsequently summarizes them.

In more detail, upon receiving a user query related to feedback, the planner decomposes the overall request into multiple sub-tasks,

forming an initial plan for task completion. The planner also leverages ICL, which includes several demonstrated examples tailored to feedback analysis. It's important to note that the initial plan may be overly detailed, potentially leading to too many execution steps and inefficiency. The planner then reflects on its initial plan by analyzing dependencies in the sub-tasks and merges them if necessary, resulting in a more concise final plan.

Once the plan is finalized, the planner assigns each sub-task with specific code generator (CG) queries to generate executable codes. It receives the execution results from the code executor and decides whether the current results satisfactorily answer the user's query. If not, the planner updates its plan or requests additional information from users if the original query is deemed ambiguous. Subsequently, it reassigns the new sub-task to CGs, repeating this process until the plan is completed and the question is answered satisfactorily. The planner then summarizes the results, providing textual summaries, code, or images generated by codes to the users. This multi-modal output is essential for feedback analysis, as it offers insights from different perspectives, providing a comprehensive and user-friendly interface necessary for many tasks [57–59].

Note that if the user is unhappy with the answer, they can provide the planner with additional instructions or even ask follow-up questions. The chat history is retained for the planner to improve or complete follow-up tasks. This iterative feedback loop contributes to the ongoing improvement and adaptability of the QA system.

3.4.2 Code Generator. The Code Generator (CG) is engineered to leverage Language Models (LLMs) for the automatic generation of Python code snippets based on tasks assigned by the planner. It can utilize common Python tools or libraries, as well as plugins tailored to feedback analysis, to offer comprehensive analytical capabilities. Feedback plugins can take various forms, such as an API call, a software module, a customized algorithm, or a machine learning model, as long as they can be invoked by a function call. The CG also utilizes In-Context Learning (ICL) and is provided with self-defined examples in a Chain-Of-Thoughts (CoT) [54] format for demonstration to: *(i)* Teach the CG to complete tasks in a predefined format. *(ii)* Provide demonstrations of feedback analysis plugins. These contextual pieces of information significantly improve the performance of the CG.

The CG is also designed with self-reflection [60] to rectify code errors during execution. If the code execution process encounters an exception or fails verification, the CG can initiate a re-generation of the code with the exception message for reference, attempting to correct the issues. The CG will attempt the re-generation process a maximum of three times. If violations persist, the CG will notify the planner of its failure to generate compliant code. This iterative process ensures that the agent maintains a robust and adaptable approach to task execution, enhancing the reliability and efficiency of the framework.

3.4.3 Code Executor. The Code Executor (CE), implemented based on Python Jupyter [61], receives the code generated by the Code Generator (CG) and collects dependent modules and plugins for execution. The choice of using Jupyter is driven by the consideration that during feedback analysis, users often refine their queries and ask follow-up questions step-by-step. This iterative process involves multiple interactions and requires the maintenance of the state of code execution throughout the entire session. This closely aligns with the programming paradigm of Jupyter Notebooks, where users run code snippets in a sequence of cells, and the program's internal state progresses sequentially. Consequently, the CE converts each user request into one or more code snippets in each round, depending on the specific plan. This approach ensures a seamless and iterative interaction model for users.

After the execution is completed, the CE preserves contextual information and sends it back to the planner along with the execution result. This information includes:

- **Logs**: Contains stdout/stderr output and log messages recorded using the logging utility within the plugins.
- **Output**: Includes all outputs of the executed Jupyter cell.
- **Artifacts**: Comprises output files such as tables and images, which are displayed in markdown format and can be downloaded via the provided URL.

The planner then organizes and summarizes all these results to respond to the user's query, presenting them in multiple formats, which completes the overall task.

Note that executing freely generated code can introduce security risks that jeopardize the system. In consideration of this, the CE is isolated and unable to access files or other processes beyond the OS user's scope. This strict confinement prevents any malicious behavior that may be attempted by users, ensuring the security and integrity of the system.

With the LLM-based QA Agents, AllHands can effectively and efficiently operate on raw feedback text and topics extracted from the initial stages. It accepts users' natural language queries and produces multi-modal responses presented to users. This ensures comprehensive insight extraction views for feedback, providing rich information to support different tasks, and thus enabling a truly "ask me anything" capability for feedback analysis.

4 SYSTEM EVALUATION

In this section, we focus on the assessment of AllHands across three dimensions, with the objective of addressing the subsequent research questions (RQs):

(1) **RQ1**: How does AllHands perform in feedback classification?
(2) **RQ2**: How does AllHands advance in performing abstractive topic modeling on verbatim feedback?
(3) **RQ3**: Can AllHands effectively respond to a varied array of questions posed in natural language, based on extensive verbatim feedback datasets?

We provide answers to these questions in the following subsections. AllHands employs GPT-3.5 [62] and GPT-4 [37] as the base LLMs at each stage, calling them through the Python API provided by OpenAI. The experiment setup for each task is detailed in their corresponding subsections.

4.1 Datasets

We collected three datasets across diverse domains to evaluate the performance of AllHands at different phases, namely Google-StoreApp [63], ForumPost [64], and MSearch. An overview of each dataset is presented in Table 1. Specifically,

Table 1: An overview of dataset employed in AllHands.

Dataset	Platform	Num. of app	Language	Label set	Size
GoogleStoreApp	Google Play Store reviews	3	English	Informative, Non-informative	11,340
ForumPost	VLC/Firefox forum posts	2	English	18 RE categories, *e.g.*, User setup, Apparent bug, *etc.*	3,654
MSearch	Search engine	1	Mixture	Actionable, Non-actionable	4,117

- **GoogleStoreApp [63]**: This dataset gathers reviews for four Android apps from Google Play, namely SwiftKey Keyboard, Facebook, Temple Run 2, and Tap Fish. Each review is manually labeled as informative and non-informative, making it suitable for the classification task.
- **ForumPost [64]**: The ForumPost dataset comprises large-scale user posts on the VLC media player and Firefox web browser. The reviews are categorized into 19 requirement engineering (RE) related categories by humans.
- **MSearch**: This dataset collects multilingual user feedback on a search engine, representing their user experience. The feedback is labeled as either actionable or non-actionable for follow-up by developers.

Note that GoogleStoreApp and ForumPost are publicly available, while MSearch is a private dataset.

Table 2: Accuracy comparison of feedback classification of AllHands with different GPT variants and other baselines.

Model	GoogleStoreApp	ForumPost	MSearch
BERT	79.8%	81.0%	61.6%
DistilBERT	72.6%	79.2%	53.0%
ALBERT	78.6%	79.1%	61.0%
RoBERTa	82.6%	80.2%	51.8%
XLM-RoBERTa	82.1%	80.3%	68.3%
AllHands			
GPT-3.5, zero-shot	77.2%	58.3%	50.1%
GPT-3.5, few-shot	82.0%	79.3%	69.2%
GPT-4, zero-shot	81.7%	67.2%	60.6%
GPT-4, few-shot	**85.7%**	**86.0%**	**77.7%**

4.2 Feedback Classification (RQ1)

First, we assess the performance of feedback classification for AllHands, leveraging the three human-labeled datasets mentioned above. This step provides additional features in a predefined dimension for feedback analysis and constitutes a crucial stage for the subsequent QA tasks.

4.2.1 Experiment Setup. We compare the performance of AllHands against a wide range of state-of-the-art transformer-based text classification baselines, namely,

- **BERT [17]**: BERT is a transformer-based model that introduces bidirectional context understanding by training on both left and right context words.
- **DistilBERT [65]**: DistilBERT is a distilled version of BERT, designed to be computationally more efficient while retaining much of BERT's performance, by employing knowledge distillation during pre-training.
- **ALBERT [66]**: ALBERT is an optimized variant of BERT that improve model scalability by introducing cross-layer parameter sharing and factorized embedding parameterization.
- **RoBERTa [67]**: RoBERTa is a refinement of the BERT model, incorporating improvement such as dynamic masking during pre-training, larger mini-batches, and removal of the next-sentence prediction objective.
- **XLM-RoBERTa [68]**: XLM-RoBERTa is a cross-lingual pre-trained language model that extends RoBERTa's architecture to handle multiple languages, making it particularly useful for multilingual feedback analysis.

In our experimentation, we utilize base-sized models as baselines, subjecting each model to fine-tuning across all layers except the embeddings. The implementation is carried out using the `PyTorch` framework [69], and the fine-tuning process is executed on a NVIDIA A100 GPU. Furthermore, in the case of AllHands, we conduct a comparative analysis between its GPT-3.5 and GPT-4 versions for classification, exploring both zero-shot (no examples) and few-shot (examples given) configurations. We utilize 10 shots for the GoogleStoreApp dataset, as it is considered simpler. For the ForumPost and MSearch datasets, we opt for 30 shots to provide a more comprehensive demonstration.

The datasets undergo a partitioning process, allocating 70% for training and validation purposes, and reserving the remaining 30% for testing. Note that for the ForumPost dataset, we exclusively consider the top 10 most prevalent labels for classification. The remaining minority categories are amalgamated into an "others" class due to their limited data and diminished significance in the analysis. We employ classification accuracy as the performance indicator.

4.2.2 Performance. Table 2 presents the accuracy performance of AllHands in feedback classification using different GPT variants, alongside other baselines, across the three datasets. Notably, GPT-4 with few-shot learning emerges as the most robust model, consistently outperforming other baselines across all datasets. XLM-RoBERTa exhibits relatively strong performance compared to smaller models, particularly on the multilingual MSearch dataset, showcasing its proficiency in multilingual learning. Despite DistilBERT's emphasis on lightweight and efficiency, it compromises performance, generally achieving lower accuracy.

As anticipated, GPT-4 outperforms GPT-3.5, underscoring its superior capabilities. Additionally, the performance of both GPT models is enhanced with few-shot learning, where provided examples contribute valuable contextual information, aiding the models in better understanding the background and objectives of the task, thereby yielding more accurate predictions. This few-shot setting is consistently applied in various stages of AllHands and has become a standard practice.

Note that the advantages of employing LLMs in AllHands extend beyond prediction accuracy. LLMs, without the need for fine-tuning, demonstrate superior generalization to diverse feedback collected from various platforms, domains, languages, and labels through ICL. This scalability positions LLMs as a robust feedback classification solution and a foundational component of AllHands, laying the groundwork for subsequent feedback QA tasks.

4.3 Abstractive Topic Modeling (RQ2)

This phase of evaluation focuses on assessing the performance of AllHands in abstractive topic modeling, which serves as an additional dimension for the QA tasks.

4.3.1 Experiment Setup. We conduct a comparative analysis of AllHands's performance against several advanced topic modeling techniques using three datasets:

- **LDA [70]**: LDA probabilistically assigns words to topics and documents to distributions of topics, unveiling hidden thematic structures in text corpora.
- **HDP [71]**: Hierarchical Dirichlet Processes (HDP) is a Bayesian nonparametric model that automatically infers the number of topics or mixture components from the data.
- **NMF [72]**: Non-negative matrix factorization (NMF) factorizes a matrix of word frequencies into two non-negative matrices, one representing topics and the other representing the distribution of topics in documents.
- **ProdLDA [73]**: ProdLDA extends LDA by incorporating a neural network-based topic model into the generative process, allowing for more flexibility and better capturing of dependencies between words and topics.
- **CTM [74]**: Contextualized Topic Modeling (CTM) extends ProdLDA by using pre-trained language representations to support topic modeling.

For these baselines, we utilize T5 [75] to summarize the keywords of each topic and the original feedback into human-readable topic labels consisting of 2-5 words. The number of topics for the baseline models is configured to match the scale of those extracted by AllHands. For AllHands, we compare its GPT-3.5 and GPT-4 versions, and its performance with or without the human-in-the-loop refinement (HITLR). Regarding the evaluation metric, we initially select BARTScore [52] to assess the similarity between the original feedback and the abstractive topic labeled by each method, as abstractive topic modeling essentially involves a summarization task where BARTScore is an effective evaluation measure. Additionally, we evaluate pairwise coherence [76] to gauge the consistency of each topic using their top-10 keywords, and the ratio of unclassified feedback labeled as "others" to (OthersRate) evaluate the model's ability to handle outliers.

4.3.2 Performance. Table 3 presents the performance comparison across all three datasets in the aforementioned dimensions. It is evident that AllHands achieves remarkable performance across all datasets, irrespective of the LLM models used or the inclusion of HITLR, consistently outperforming other baselines. Particularly, these baselines generally exhibit lower BARTScores, attributable to their utilization of smaller models (T5) for summarizing each feedback label. Moreover, we observe that AllHands achieves significantly higher coherence, indicating that topics extracted by LLMs exhibit semantic similarity between words, rendering them more interpretable and meaningful. Additionally, we note that AllHands tends to classify fewer feedback instances as "others", indicating its ability to better handle and incorporate outliers due to the remarkable summarization ability of LLMs. This is advantageous in topic modeling, as these "others" may also contain valuable insights from the feedback, which AllHands is adept at capturing.

When considering the ablation comparison within the AllHands method, which involves varying the LLM models used or the inclusion of HITLR, we observe two notable phenomena. Firstly, employing GPT-4 generally yields superior performance compared to GPT-3.5. This aligns with our expectations, given the overall stronger capabilities of GPT-4. However, the margin between the two models is not substantial. This suggests that utilizing GPT-3.5 may suffice for the task of abstractive topic modeling, especially considering its outperformance of other baselines, despite the higher resource costs associated with GPT-4. Secondly, the inclusion of HITLR leads to a significant enhancement in performance for both GPT-3.5 and GPT-4. This underscores the importance of integrating human knowledge into the topic modeling process, as LLMs alone may generate less meaningful topics. Incorporating human insights can markedly improve the quality of the generated labels.

4.3.3 Case Study. To provide further insights into how AllHands excels in topic abstraction, Table 4 illustrates the extracted topic labels for three feedback instances from each dataset using AllHands with GPT-4 and HITLR, alongside the best baseline CTM. A comparative analysis reveals several advantages of AllHands in abstractive topic modeling. Firstly, AllHands is capable of summarizing multiple meaningful topic labels for a single feedback, whereas CTM only provides one label for each. This capability is desirable, as feedback often encompasses various relevant topics, all of which can be effectively captured by LLMs. Secondly, the topic labels generated by AllHands exhibit fewer instances of hallucination. For instance, for the feedback "please make Windows 10 more stable", AllHands summarizes it as "feature request; reliability", while CTM erroneously labels it as "Minecraft Windows", which is entirely unrelated. Such occurrences are frequent in other feedback instances as well, indicating that AllHands can offer more reliable topic labels. Lastly, AllHands provides more meaningful and general topic labels. For example, when labeling "A taskbar item is created and takes up space in the taskbar", AllHands categorizes it as "UI/UX; functionality or feature issue", whereas CTM assigns it the label "add bookmarks toolbar", which is overly specific and results in numerous scattered topics, complicating data analysis efforts.

The experimental results and case study collectively indicate that AllHands, utilizing LLMs as a revolutionary approach compared to traditional topic modeling methods, excels in autonomously summarizing topics of interest from unstructured verbatim feedback. These extracted topics serve as a crucial foundation for subsequent QA tasks.

Table 3: The performance comparison of the abstractive topic modeling task. Best results are highlighted with bold.

Method	GoogleStoreApp			ForumPost			MSearch		
	BARTScore	Coherence	OtherRate	BARTScore	Coherence	OthersRate	BARTScore	Coherence	OthersRate
LDA	-7.429	0.001	14%	-6.837	0.02	6%	-7.092	0.01	25%
HDP	-7.473	0.003	15%	-7.002	0.01	4%	-7.359	0.004	22%
NMF	-7.523	0.008	13%	-6.984	0.018	3%	-7.16	0.007	21%
ProdLDA	-6.925	0.017	13%	-6.848	0.012	4%	-6.824	0.008	22%
CTM	-7.113	0.031	11%	-6.733	0.024	4%	-7.038	0.013	20%
AllHands									
GPT-3.5 w/o HITLR	-6.914	0.028	**7%**	-6.942	0.029	12%	-6.679	0.019	16%
GPT-3.5 w/ HITLR	**-6.822**	0.025	**7%**	-6.557	0.037	4%	-6.426	0.027	15%
GPT-4 w/o HITLR	-7.007	0.044	**7%**	-6.72	0.033	3%	-6.68	0.018	17%
GPT-4 w/ HITLR	-6.899	**0.046**	**7%**	**-6.628**	**0.038**	**2%**	**-6.242**	**0.030**	**11%**

Table 4: The examples topic labels summarized by AllHands and CTM in three datasets.

Dataset	Feedback	AllHands	CTM
GoogleStoreApp	bring back the cheetah filter it's all I looked forward to in life please and thank you	feature request	bring back bunny face filter
	your phone sucksssssss there goes my data cap because your apps suck	insult; functionality or feature issue	whatsapp not working
	please make windows 10 more stable.	feature request; reliability	minecraft windows
ForumPost	I have followed these instructions but I still dont get spell check as I write.	spell checking feature	dictionary
	A taskbar item is created and takes up space in the taskbar.	UI/UX; functionality or feature issue	add bookmarks toolbar
	Chrome loads pages without delay on this computer.	Chrome; performance	self signed certificate
MSearch	It is not the model of machine that I have indicated.	incorrect or wrong information	gremio
	Wrong car model	incorrect or wrong information	misspelled image
	not gives what im asking for	unhelpful or irrelevant results	asking questions

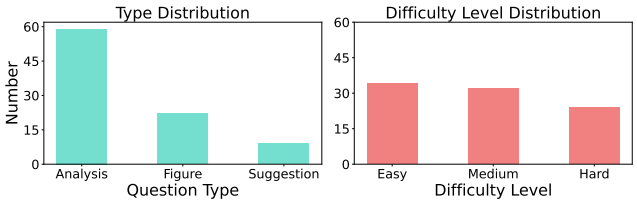

Figure 7: The questions distributions on types and difficulties.

4.4 Free-style QA (RQ3)

Finally, we employ AllHands to respond to users' inquiries in natural language using three feedback datasets and assess the quality of its responses, thereby achieving the overarching objective of AllHands.

4.4.1 Questions Design. To comprehensively evaluate the QA performance of AllHands, we engaged three data scientists to design 30 questions, commonly used in their daily feedback analysis or gathered from existing literature. These questions were aimed at gaining insights from each dataset. These questions cover commonly asked topics for feedback analysis and comprise a total of 90 questions, categorized into three types:

- **Analysis**: These questions seek specific statistical information about the feedback topics or verbatim for analytical purposes.
- **Figure**: These questions request the creation of various visualizations, such as figures or charts, to represent the statistics of feedback topics or verbatim.
- **Suggestion**: These open-ended questions prompt respondents to provide suggestions for product improvement based on the statistical analysis of feedback topics or verbatim.

By including these three types of questions, commonly used in data analysis for verbatim feedback, we aim to comprehensively evaluate AllHands's performance.

Additionally, we classified each question into three levels of difficulty, namely easy, medium, and hard based on multidimensional criteria. These criteria include:

- **Number of Steps**: The number of steps required to complete the task.
- **Number of Filters**: The number of filters needed to apply to the data.
- **Plotting a Figure**: Whether the question involves plotting a figure.
- **Use of Out-of-scope Filters**: Whether the query requires the use of filters beyond the existing columns in the data.
- **Open-ended Nature**: Whether the question is open-ended, requiring comprehensive data analysis to provide a suggestion.

We weighted these five factors to label each question into one of the three difficulty levels. This classification enables us to evaluate how AllHands handles requests of varying complexity. We present the overall distribution of question types and difficulty level in Fig. 7. Detailed lists of questions on the three dataset are presented in Table 5, 6 7 in the supplementary material.

4.4.2 Evaluation Metric. We assess the quality of each response generated by AllHands along three dimensions: *(i)* comprehensiveness, *(ii)* correctness, and *(iii)* readability. Each metric is graded on a scale from 1 to 5, representing low to high quality. Specifically, comprehensiveness assesses whether the response reflects the extent to which the answer covers all relevant aspects of the task and utilizes diverse formats effectively to provide a comprehensive understanding:

- **Low (1)**: The response lacks completeness. It fails to utilize various output modalities effectively.
- **Limited (2)**: The answer is somewhat complete, but its comprehensiveness is restricted, and it lacks diversity in output modalities, limiting insights.
- **Moderate (3)**: The response includes moderately complete information, contributing to the task, but there's room for improvement in both completeness and diversity of output modality.
- **High (4)**: The information is quite comprehensive, providing valuable insights. It utilizes diverse output modalities effectively, enriching the response.
- **Very High (5)**: The answer is exceptionally comprehensive, offering thorough insights. It utilizes a wide range of output modalities exceptionally well, exceeding expectations.

Correctness evaluates the accuracy and relevance of the information provided, assessing whether the answer contains errors, especially in code, tables, or images, and whether it aligns with the task requirements:

- **Inaccurate (1)**: The response contains significant errors, including code, table, or image errors, leading to a complete misinterpretation of the task. It's irrelevant to the given context.
- **Partially Correct (2)**: Some elements of the answer are accurate, but overall, the response contains substantial errors in code, table, or image, impacting its relevance.
- **Mostly Correct (3)**: The majority of the information is accurate, but there are noticeable errors in code, table, or image that may affect the overall understanding and relevance to the task.
- **Correct (4)**: The response is mostly accurate, with minor errors in code, table, or image that do not significantly impact the overall correctness and relevance of the information.
- **Completely Correct (5)**: The answer is entirely accurate, with no discernible errors in code, table, or image, demonstrating a high level of precision, relevance, and reliability.

In addition, readability evaluates the clarity and ease of understanding of the answer, considering factors such as organization, language clarity, and the quality and presentation of images. Specifically:

- **Unintelligible (1)**: The answer is extremely difficult to understand, with poor organization, unclear expression of ideas, and low-quality images.
- **Difficult to Follow (2)**: The response is somewhat challenging to follow, requiring effort to decipher due to unclear structure or language. The quality and presentation of images are suboptimal.
- **Moderately Readable (3)**: The answer is generally clear, but there are areas where improved clarity in expression or organization is needed. The quality of images is acceptable.
- **Clear (4)**: The information is presented in a clear and well-organized manner, making it easy for the reader to follow and understand. The quality and presentation of images are good.
- **Exceptionally Clear (5)**: The answer is exceptionally clear, with precise and well-structured presentation. The quality and presentation of images are excellent, enhancing overall readability and comprehension.

To ensure fair scoring, we recruited 10 survey participants with backgrounds in data science to assess the comprehensiveness, correctness, and readability according to the criteria outlined in Sec. **??**.

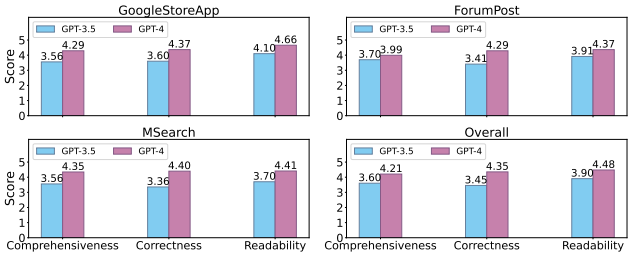

Figure 8: Answer quality assessment by humans of the QA agent employed in AllHands.

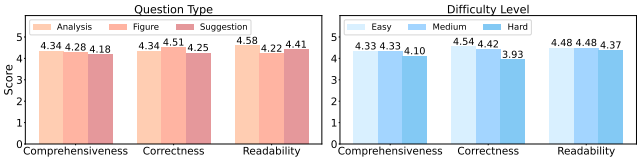

Figure 9: Answer quality assessment comparison across questions types and difficulty levels for the QA agent using GPT-4.

Each participant was randomly assigned 27 questions to evaluate the responses generated by both the GPT-3.5 and GPT-4 versions of the QA agent in AllHands. The names of the GPT models were concealed to prevent bias. Each question's response was independently scored by 3 participants, and the average scores were calculated for reliability. Ultimately, we collected a total of 270 scores, with 3 scores for each question.

4.4.3 Performance Comparison. First, let's compare the assessment of answer quality by survey participants across the three dimensions of the three datasets and their average, as shown in Fig. 8. It is evident that the QA agent employed in AllHands demonstrates notable performance across all evaluated dimensions, irrespective of the model used. Across all datasets and dimensions, the agent achieves an average score of over 3, indicating its proficient performance in analyzing feedback data. Particularly noteworthy is the consistently high performance of its GPT-4 version, which consistently scores over 4 across all datasets in terms of comprehensiveness, correctness, and readability of its answers. Given that a score of 4 represents a high standard in our scoring system, this suggests that AllHands, particularly when equipped with GPT-4, adeptly serves as a feedback analytic tool and significantly reduces the need for human intervention by providing natural language responses to user queries in a revolutionary manner.

Furthermore, its GPT-4 version consistently outperforms GPT-3.5 by 16.9% in comprehensiveness, 26.1% in correctness, and 14.9% in readability. This substantial performance gap underscores the disparities in capabilities between LLM models. GPT-3.5 often struggles to generate accurate code and overlooks certain details during the analysis process, rendering it suboptimal for this task.

In Fig. 9, we delineate the average assessment scores of the QA agent employing GPT-4 across three datasets, categorized by question types and difficulty levels. Notably, we observe that AllHands tends to provide more comprehensive responses to analysis and figure-related queries compared to suggestion queries. This observation aligns with expectations, as analysis and figure-related questions typically involve deterministic processes, whereas suggestions

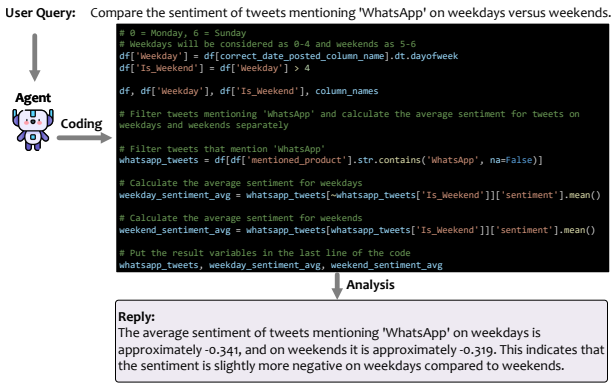

Figure 10: A case study on an analysis-related user query.

are more open-ended. Consequently, the agent may overlook certain aspects when providing suggestions. This trend is also reflected in the correctness dimension, as suggestion-related answers are more subjective and complex, demanding a comprehensive understanding of the data across all dimensions. These factors may contribute to suboptimal answers for the agent in suggestion-related queries. Conversely, we note that figure-related questions achieve the lowest readability scores. This could be attributed to instances where the agent fails to optimize the layout of generated figures, such as using excessively small font sizes, thereby compromising the visual clarity of the figures.

Taking a closer look at the right subplot of Fig. 9, which illustrates the comparison across different difficulty levels, we observe a consistent trend where the average scores decrease with increasing difficulty, as anticipated. Questions with higher difficulty levels are inherently more complex and demand a more comprehensive understanding of the data to provide accurate responses. This explains why the comprehensiveness and correctness scores are notably lower for hard questions. The readability, however does not drop significantly for hard questions, indicating that the QA agent consistently delivers readable answers to all queries.

4.4.4 Case Study. Finally, we present some case studies of the QA Agent to illustrate how AllHands effectively handles user queries in natural language for feedback analysis and provides insightful answers.

Case 1: Compare the sentiment of tweets mentioning 'WhatsApp' on weekdays versus weekends.

In Fig. 10, we demonstrate how our AllHands system responds to an analysis-related user query. The agent proficiently translates the query into Python code, wherein it first partitions the data into weekdays and weekends, then filters the data related to 'WhatsApp', and finally compares the difference between these two categories. The response provided is highly readable, comprehensive, and correct, showcasing the system's remarkable ability to fulfill analysis-related user requests. This is achieved through sophisticated data manipulation using the `Pandas` library.

Figure 11: A case study on a figure-related user query.

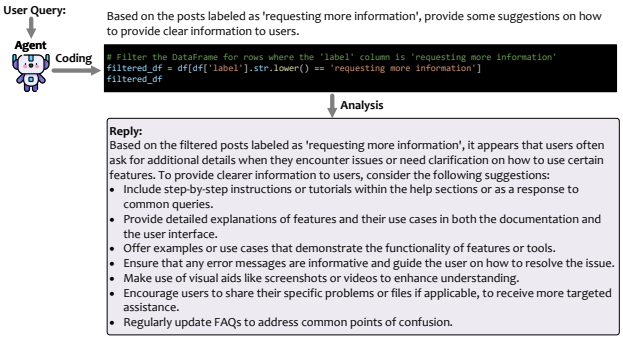

Figure 12: A case study on a suggestion-related user query.

Case 2: Draw an issue river for top 7 topics.

In Fig. 11, we present a different scenario to illustrate how AllHands can generate a issue river [57] in response to a user query. The `issue_river` function, integrated as a plugin within the agent, is utilized to accomplish this task. The agent accurately filters the data and produces a visually appealing issue river plot, reminiscent of the one described in [57]. This demonstrates AllHands's capability to effectively utilize pre-defined tools to fulfill complex requests and generate aesthetically pleasing plots, highlighting its remarkable ability to provide comprehensive responses with multi-modality.

Case 3: Based on the posts labeled as 'requesting more information', provide some suggestions on how to provide clear information to users.

Finally, we demonstrate how AllHands can address open-ended questions by providing suggestions for product improvement, as

depicted in Fig. 12. The agent begins by filtering the necessary data and thoroughly analyzing it. Subsequently, AllHands offers seven highly comprehensive and insightful suggestions, all of which are highly relevant to the topic at hand. This response receives average scores of 5, 4, and 5 for the three evaluation dimensions, affirming AllHands's ability to effectively handle such open-ended questions and provide developers with insightful recommendations for product enhancement based on feedback data analysis.

Overall, these three cases demonstrate that AllHands is adept at responding to user queries in natural language and providing insightful answers for various types of data. This capability allows AllHands to truly embody the "Ask me anything" paradigm for feedback data analysis, thereby revolutionizing the traditional methods in this field.

5 THREATS TO VALIDITY

5.1 Internal Validity

The output of AllHands for identical queries may exhibit variability owing to the inherent instability of LLM outputs. This instability stems from continuous upgrades to LLM services involving model versions and API modifications. In response to this challenge, we strategically adjust hyperparameters, such as temperature and top_p, setting them to zero. This minimizes the variance in LLM responses, thereby maximizing the reproducibility of experimental results.

Additionally, the internal dataset utilized in this study is derived from feedback on a search engine. Given the substantial volume of feedback, we restrict our analysis to data from the most recent two months, prioritizing significance and relevance. However, this selective timeframe may introduce variance in the experiments, particularly in the realms of classification and topic modeling.

5.2 External Validity

Although our primary emphasis lies in feedback analysis, the methodology employed in our work exhibits seamless applicability to diverse textual data sources. This includes user-generated content in social media, serving analytical and QA purposes, all without necessitating model fine-tuning. Users have the flexibility to specify the domain or direction for topic modeling based on their interests and can create custom plugins to cater to ad-hoc QA demands.

Note that the questions posed to AllHands must align with the contextual relevance of the feedback data to yield satisfactory answers. In instances where questions are unrelated to the available data context, there is a risk of answer rejection, as the required information cannot be derived from the provided dataset.

6 RELATED WORK

In this section, we overview relevant research and practice research and practical applications within the domains of user feedback mining and LLMs utilization in the context of software engineering.

6.1 Users Feedback Mining

The exploration of user feedback mining has emerged as a crucial area of investigation within both software engineering and natural language processing [15, 16, 77–81]. Chen *et al.*, introduce AR-miner, employing topic modeling techniques to extract insightful reviews, thereby offering valuable perspectives for developers [63]. Gao *et al.*, developed DIVER [29], a tool designed to identify emerging issues in WeChat through topic modeling, thereby enhancing the stability and efficiency of the conventional IDEA approach [57]. Additionally, Gao *et al.*, presented SOLAR, a system capable of summarizing useful user reviews by integrating helpfulness and topic-sentiment modeling, along with multi-factor ranking [82]. Ebrahimi *et al.*, delved into privacy concerns within mobile app reviews. Their research involved term extraction in the privacy domain and text summarization to underscore prominent concerns across top aspects [59].

Our proposed AllHands districts from the existing endeavors by offering a comprehensive approach that spans classification, topic modeling, and the provision of high-quality free-style question answering for extensive verbatim feedback datasets. This consolidated approach serves as a one-stop solution for diverse feedback mining requirements.

6.2 LLMs for Software Requirement Engineering

The advent of LLMs is catalyzing a paradigm shift in the field of software engineering, enhancing both feasibility and capabilities within the domain [83–85], especially in the field of requirement engineering [86]. Arora *et al.*, articulate a visionary perspective on LLM-driven requirement engineering, delineating several promising directions that leverage LLMs across requirement elicitation, specification, analysis, and validation [87]. Research by Abualhaija *et al.*, introduces an automated QA approach tailored for compliance requirements with language models [88]. Furthermore, Zhang *et al.*, propose PersonaGen, a system utilizing GPT-4 and knowledge graphs to generate personas from user feedback, thereby enhancing the comprehension of user requirements [89].

Diverging from the aforementioned initiatives, AllHands introduces an innovative system that harnesses the potency of LLMs for holistic feedback analysis and question answering.

7 CONCLUSION

This paper introduces AllHands, an innovative and comprehensive analysis framework tailored for large-scale verbatim feedback analysis, featuring a natural language interface and harnessing the capabilities of LLMs. AllHands initiates the analysis pipeline by transforming textual feedback into an augmented structural format through classification and abstractive topic modeling. Notably, LLMs are integrated to enhance the accuracy, robustness, generalization, and user-friendliness of conventional approaches. Subsequently, AllHands integrates an LLM agent to facilitate a natural language interface, translating users' inquiries into Python code executed on the feedback. This facilitates multi-modal responses, encompassing text, code, tables, and images. Evaluation conducted on both offline datasets in diverse domains consistently demonstrates the superior performance of AllHands. It provides users with a flexible, extendable, and "ask me anything' experience, marking a pioneering advancement in future feedback analysis frameworks.

ACKNOWLEDGMENT

We extend our sincere gratitude to Jue Zhang for his invaluable contributions to our research endeavor. Jue Zhang's insightful input and engaging discussions have significantly enriched our exploration into the realm of abstract topic modeling.

REFERENCES

[1] Daniela Oelke, Ming Hao, Christian Rohrdantz, Daniel A Keim, Umeshwar Dayal, Lars-Erik Haug, and Halldór Janetzko. Visual opinion analysis of customer feedback data. In *2009 IEEE symposium on visual analytics science and technology*, pages 187–194. IEEE, 2009.

[2] Prashant Pandey, Nitasha Soni, et al. Sentiment analysis on customer feedback data: Amazon product reviews. In *2019 International conference on machine learning, big data, cloud and parallel computing (COMITCon)*, pages 320–322. IEEE, 2019.

[3] Ze Shi Li, Nowshin Nawar Arony, Kezia Devathasan, Manish Sihag, Neil Ernst, and Daniela Damian. Unveiling the life cycle of user feedback: Best practices from software practitioners. *arXiv preprint arXiv:2309.07345*, 2023.

[4] Stefan Gärtner and Kurt Schneider. A method for prioritizing end-user feedback for requirements engineering. In *2012 5th International Workshop on Co-operative and Human Aspects of Software Engineering (CHASE)*, pages 47–49. IEEE, 2012.

[5] Eric Knauss, Daniel Lubke, and Sebastian Meyer. Feedback-driven requirements engineering: the heuristic requirements assistant. In *2009 IEEE 31st international conference on software engineering*, pages 587–590. IEEE, 2009.

[6] Liping Zhao, Waad Alhoshan, Alessio Ferrari, Keletso J Letsholo, Muideen A Ajagbe, Erol-Valeriu Chioasca, and Riza T Batista-Navarro. Natural language processing for requirements engineering: A systematic mapping study. *ACM Computing Surveys (CSUR)*, 54(3):1–41, 2021.

[7] Eriks Klotins, Michael Unterkalmsteiner, and Tony Gorschek. Software engineering in start-up companies: An analysis of 88 experience reports. *Empirical Software Engineering*, 24:68–102, 2019.

[8] Chao-Hong Liu, Declan Groves, Akira Hayakawa, Alberto Poncelas, and Qun Liu. Understanding meanings in multilingual customer feedback. *arXiv preprint arXiv:1806.01694*, 2018.

[9] Stephen Withall. *Software requirement patterns*. Pearson Education, 2007.

[10] Sebastiano Panichella, Andrea Di Sorbo, Emitza Guzman, Corrado A Visaggio, Gerardo Canfora, and Harald C Gall. How can i improve my app? classifying user reviews for software maintenance and evolution. In *2015 IEEE international conference on software maintenance and evolution (ICSME)*, pages 281–290. IEEE, 2015.

[11] Dennis Pagano and Walid Maalej. User feedback in the appstore: An empirical study. In *2013 21st IEEE international requirements engineering conference (RE)*, pages 125–134. IEEE, 2013.

[12] Des Greer and Guenther Ruhe. Software release planning: an evolutionary and iterative approach. *Information and software technology*, 46(4):243–253, 2004.

[13] Giovanni Grano, Andrea Di Sorbo, Francesco Mercaldo, Corrado A Visaggio, Gerardo Canfora, and Sebastiano Panichella. Android apps and user feedback: a dataset for software evolution and quality improvement. In *Proceedings of the 2nd ACM SIGSOFT international workshop on app market analytics*, pages 8–11, 2017.

[14] Maryam Edalati, Ali Shariq Imran, Zenun Kastrati, and Sher Muhammad Daudpota. The potential of machine learning algorithms for sentiment classification of students' feedback on mooc. In *Intelligent Systems and Applications: Proceedings of the 2021 Intelligent Systems Conference (IntelliSys) Volume 3*, pages 11–22. Springer, 2022.

[15] Hamed Jelodar, Yongli Wang, Chi Yuan, Xia Feng, Xiahui Jiang, Yanchao Li, and Liang Zhao. Latent dirichlet allocation (lda) and topic modeling: models, applications, a survey. *Multimedia Tools and Applications*, 78:15169–15211, 2019.

[16] Cuiyun Gao, Jichuan Zeng, David Lo, Chin-Yew Lin, Michael R Lyu, and Irwin King. Infar: Insight extraction from app reviews. In *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 904–907, 2018.

[17] Jacob Devlin Ming-Wei Chang Kenton and Lee Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, pages 4171–4186, 2019.

[18] Nikolaos Giarelis, Charalampos Mastrokostas, and Nikos Karacapilidis. Abstractive vs. extractive summarization: An experimental review. *Applied Sciences*, 13(13):7620, 2023.

[19] Sewon Min, Xinxi Lyu, Ari Holtzman, Mikel Artetxe, Mike Lewis, Hannaneh Hajishirzi, and Luke Zettlemoyer. Rethinking the role of demonstrations: What makes in-context learning work? *arXiv preprint arXiv:2202.12837*, 2022.

[20] Chien Chin Chen and You-De Tseng. Quality evaluation of product reviews using an information quality framework. *Decision Support Systems*, 50(4):755–768, 2011.

[21] Walid Maalej, Zijad Kurtanović, Hadeer Nabil, and Christoph Stanik. On the automatic classification of app reviews. *Requirements Engineering*, 21:311–331, 2016.

[22] Nathan Baker. Employee feedback technologies in the human performance system. *Human Resource Development International*, 13(4):477–485, 2010.

[23] Jipeng Qiang, Zhenyu Qian, Yun Li, Yunhao Yuan, and Xindong Wu. Short text topic modeling techniques, applications, and performance: a survey. *IEEE Transactions on Knowledge and Data Engineering*, 34(3):1427–1445, 2020.

[24] Ike Vayansky and Sathish AP Kumar. A review of topic modeling methods. *Information Systems*, 94:101582, 2020.

[25] Rubens Santos, Eduard C Groen, and Karina Villela. An overview of user feedback classification approaches. In *REFSQ Workshops*, volume 3, pages 357–369, 2019.

[26] Yuening Hu, Jordan Boyd-Graber, Brianna Satinoff, and Alison Smith. Interactive topic modeling. *Machine learning*, 95:423–469, 2014.

[27] Mohammad A Hadi and Fatemeh H Fard. Evaluating pre-trained models for user feedback analysis in software engineering: A study on classification of app-reviews. *Empirical Software Engineering*, 28(4):88, 2023.

[28] Pengtao Xie and Eric P Xing. Integrating document clustering and topic modeling. In *Proceedings of the Twenty-Ninth Conference on Uncertainty in Artificial Intelligence*, pages 694–703, 2013.

[29] Cuiyun Gao, Wujie Zheng, Yuetang Deng, David Lo, Jichuan Zeng, Michael R Lyu, and Irwin King. Emerging app issue identification from user feedback: Experience on wechat. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 279–288. IEEE, 2019.

[30] Cuiyun Gao, Jichuan Zeng, Zhiyuan Wen, David Lo, Xin Xia, Irwin King, and Michael R Lyu. Emerging app issue identification via online joint sentiment-topic tracing. *IEEE Transactions on Software Engineering*, 48(8):3025–3043, 2021.

[31] Ehsan Noei, Feng Zhang, and Ying Zou. Too many user-reviews! what should app developers look at first? *IEEE Transactions on Software Engineering*, 47(2):367–378, 2019.

[32] Emitza Guzman and Walid Maalej. How do users like this feature? a fine grained sentiment analysis of app reviews. In *2014 IEEE 22nd international requirements engineering conference (RE)*, pages 153–162. Ieee, 2014.

[33] William Martin, Federica Sarro, and Mark Harman. Causal impact analysis for app releases in google play. In *Proceedings of the 2016 24th ACM SIGSOFT international symposium on foundations of software engineering*, pages 435–446, 2016.

[34] Meng Zhang, Yang Qian, Yuanchun Jiang, Yuyang Wang, and Yezheng Liu. Helpfulness prediction for vr application reviews: Exploring topic signals for causal inference. In *2022 IEEE International Symposium on Mixed and Augmented Reality Adjunct (ISMAR-Adjunct)*, pages 17–21. IEEE, 2022.

[35] Xiaozhou Li, Zheying Zhang, and Kostas Stefanidis. Mobile app evolution analysis based on user reviews. In *New Trends in Intelligent Software Methodologies, Tools and Techniques*, pages 773–786. IOS Press, 2018.

[36] Tong Li, Mingyang Zhang, Hancheng Cao, Yong Li, Sasu Tarkoma, and Pan Hui. "what apps did you use?": Understanding the long-term evolution of mobile app usage. In *Proceedings of the web conference 2020*, pages 66–76, 2020.

[37] OpenAI. GPT-4 technical report, 2023.

[38] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[39] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 11 2019.

[40] Xuanhe Zhou, Chengliang Chai, Guoliang Li, and Ji Sun. Database meets artificial intelligence: A survey. *IEEE Transactions on Knowledge and Data Engineering*, 34(3):1096–1116, 2020.

[41] Baoli Li and Liping Han. Distance weighted cosine similarity measure for text classification. In *Intelligent Data Engineering and Automated Learning–IDEAL 2013: 14th International Conference, IDEAL 2013, Hefei, China, October 20-23, 2013. Proceedings 14*, pages 611–618. Springer, 2013.

[42] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[43] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. Albert: A lite bert for self-supervised learning of language representations. *arXiv preprint arXiv:1909.11942*, 2019.

[44] Yin Zhang, Rong Jin, and Zhi-Hua Zhou. Understanding bag-of-words model: a statistical framework. *International journal of machine learning and cybernetics*, 1:43–52, 2010.

[45] Maarten Grootendorst. Bertopic: Neural topic modeling with a class-based tf-idf procedure. *arXiv preprint arXiv:2203.05794*, 2022.

[46] Thanh-Nam Doan and Tuan-Anh Hoang. Benchmarking neural topic models: An empirical study. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pages 4363–4368, 2021.

[47] Ziqiang Cao, Sujian Li, Yang Liu, Wenjie Li, and Heng Ji. A novel neural topic model and its supervised extension. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 29, 2015.

[48] Parth Mehta. From extractive to abstractive summarization: A journey. In *ACL (Student Research Workshop)*, pages 100–106. Springer, 2016.

[49] Li Zhuang, Feng Jing, and Xiao-Yan Zhu. Movie review mining and summarization. In *Proceedings of the 15th ACM international conference on Information and knowledge management*, pages 43–50, 2006.

[50] Tianyi Zhang, Faisal Ladhak, Esin Durmus, Percy Liang, Kathleen McKeown, and Tatsunori B Hashimoto. Benchmarking large language models for news summarization. *arXiv preprint arXiv:2301.13848*, 2023.

[51] Daniel Müllner. Modern hierarchical, agglomerative clustering algorithms. *arXiv preprint arXiv:1109.2378*, 2011.

[52] Weizhe Yuan, Graham Neubig, and Pengfei Liu. Bartscore: Evaluating generated text as text generation. *Advances in Neural Information Processing Systems*, 34:27263–27277, 2021.

[53] Bo Qiao, Liqun Li, Xu Zhang, Shilin He, Yu Kang, Chaoyun Zhang, Fangkai Yang, Hang Dong, Jue Zhang, Lu Wang, et al. TaskWeaver: A code-first agent framework. *arXiv preprint arXiv:2311.17541*, 2023.

[54] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022.

[55] Ruomeng Ding, Chaoyun Zhang, Lu Wang, Yong Xu, Minghua Ma, Wei Zhang, Si Qin, Saravan Rajmohan, Qingwei Lin, and Dongmei Zhang. Everything of thoughts: Defying the law of penrose triangle for thought generation. *arXiv preprint arXiv:2311.04254*, 2023.

[56] Chaoyun Zhang, Liqun Li, Shilin He, Xu Zhang, Bo Qiao, Si Qin, Minghua Ma, Yu Kang, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Qi Zhang. UFO: A UI-Focused Agent for Windows OS Interaction. *arXiv preprint arXiv:2402.07939*, 2024.

[57] Cuiyun Gao, Jichuan Zeng, Michael R Lyu, and Irwin King. Online app review analysis for identifying emerging issues. In *Proceedings of the 40th International Conference on Software Engineering*, pages 48–58, 2018.

[58] Christian Montag, Benjamin Becker, and Chunmei Gan. The multipurpose application wechat: a review on recent research. *Frontiers in psychology*, 9:2247, 2018.

[59] Fahimeh Ebrahimi and Anas Mahmoud. Unsupervised summarization of privacy concerns in mobile application reviews. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, pages 1–12, 2022.

[60] Noah Shinn, Beck Labash, and Ashwin Gopinath. Reflexion: an autonomous agent with dynamic memory and self-reflection. *arXiv preprint arXiv:2303.11366*, 2023.

[61] Lorena A Barba. The python/jupyter ecosystem: Today's problem-solving environment for computational science. *Computing in Science & Engineering*, 23(3):5–9, 2021.

[62] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.

[63] Ning Chen, Jialiu Lin, Steven CH Hoi, Xiaokui Xiao, and Boshen Zhang. AR-miner: mining informative reviews for developers from mobile app marketplace. In *Proceedings of the 36th international conference on software engineering*, pages 767–778, 2014.

[64] James Tizard, Hechen Wang, Lydia Yohannes, and Kelly Blincoe. Can a conversation paint a picture? mining requirements in software forums. In *2019 IEEE 27th International Requirements Engineering Conference (RE)*, pages 17–27. IEEE, 2019.

[65] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.

[66] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. Albert: A lite bert for self-supervised learning of language representations. In *International Conference on Learning Representations*, 2020.

[67] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Ro{bert}a: A robustly optimized {bert} pretraining approach, 2020.

[68] Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Édouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. Unsupervised cross-lingual representation learning at scale. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8440–8451, 2020.

[69] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.

[70] David M Blei, Andrew Y Ng, and Michael I Jordan. Latent dirichlet allocation. *Journal of machine Learning research*, 3(Jan):993–1022, 2003.

[71] Yee Teh, Michael Jordan, Matthew Beal, and David Blei. Sharing clusters among related groups: Hierarchical dirichlet processes. *Advances in neural information processing systems*, 17, 2004.

[72] Daniel Lee and H Sebastian Seung. Algorithms for non-negative matrix factorization. *Advances in neural information processing systems*, 13, 2000.

[73] Akash Srivastava and Charles Sutton. Autoencoding variational inference for topic models. *arXiv preprint arXiv:1703.01488*, 2017.

[74] Federico Bianchi, Silvia Terragni, Dirk Hovy, Debora Nozza, and Elisabetta Fersini. Cross-lingual contextualized topic models with zero-shot learning. *arXiv preprint arXiv:2004.07737*, 2020.

[75] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *The Journal of Machine Learning Research*, 21(1):5485–5551, 2020.

[76] Anjie Fang, Craig Macdonald, Iadh Ounis, and Philip Habel. Using word embedding to evaluate the coherence of topics from twitter data. In *Proceedings of the 39th International ACM SIGIR conference on Research and Development in Information Retrieval*, pages 1057–1060, 2016.

[77] Jacek Dkabrowski, Emmanuel Letier, Anna Perini, and Angelo Susi. Analysing app reviews for software engineering: a systematic literature review. *Empirical Software Engineering*, 27(2):43, 2022.

[78] Liu Wang, Haoyu Wang, Xiapu Luo, Tao Zhang, Shangguang Wang, and Xuanzhe Liu. Demystifying "removed reviews" in ios app store. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 1489–1499, 2022.

[79] Tse-Hsun Chen, Stephen W Thomas, and Ahmed E Hassan. A survey on the use of topic models when mining software repositories. *Empirical Software Engineering*, 21:1843–1919, 2016.

[80] Jacek Dkabrowski, Emmanuel Letier, Anna Perini, and Angelo Susi. Mining user feedback for software engineering: Use cases and reference architecture. In *2022 IEEE 30th International Requirements Engineering Conference (RE)*, pages 114–126. IEEE, 2022.

[81] Peter Devine, Yun Sing Koh, and Kelly Blincoe. Evaluating unsupervised text embeddings on software user feedback. In *2021 IEEE 29th International Requirements Engineering Conference Workshops (REW)*, pages 87–95. IEEE, 2021.

[82] Cuiyun Gao, Yaoxian Li, Shuhan Qi, Yang Liu, Xuan Wang, Zibin Zheng, and Qing Liao. Listening to users' voice: Automatic summarization of helpful app reviews. *IEEE Transactions on Reliability*, 2022.

[83] Yinfang Chen, Huaibing Xie, Minghua Ma, Yu Kang, Xin Gao, Liu Shi, Yunjie Cao, Xuedong Gao, Hao Fan, Ming Wen, Jun Zeng, Supriyo Ghosh, Xuchao Zhang, Chaoyun Zhang, et al. Empowering practical root cause analysis by large language models for cloud incidents. *arXiv preprint arXiv:2305.15778*, 2023.

[84] Pengxiang Jin, Shenglin Zhang, Minghua Ma, Haozhe Li, Yu Kang, Liqun Li, Yudong Liu, Bo Qiao, Chaoyun Zhang, Pu Zhao, et al. Assess and summarize: Improve outage understanding with large language models. *arXiv preprint arXiv:2305.18084*, 2023.

[85] Yuxuan Jiang, Chaoyun Zhang, Shilin He, Zhihao Yang, Minghua Ma, Si Qin, Yu Kang, Yingnong Dang, Saravan Rajmohan, Qingwei Lin, and Dongmei Zhang. Xpert: Empowering incident management with query recommendations via large language models. *arXiv preprint arXiv:2312.11988*, 2023.

[86] Yawen Wang, Junjie Wang, Hongyu Zhang, Xuran Ming, Lin Shi, and Qing Wang. Where is your app frustrating users? In *Proceedings of the 44th International Conference on Software Engineering*, pages 2427–2439, 2022.

[87] Chetan Arora, John Grundy, and Mohamed Abdelrazek. Advancing requirements engineering through generative ai: Assessing the role of llms. *arXiv preprint arXiv:2310.13976*, 2023.

[88] Sallam Abualhaija, Chetan Arora, Amin Sleimi, and Lionel C Briand. Automated question answering for improved understanding of compliance requirements: A multi-document study. In *2022 IEEE 30th International Requirements Engineering Conference (RE)*, pages 39–50. IEEE, 2022.

[89] Xishuo Zhang, Lin Liu, Yi Wang, Xiao Liu, Hailong Wang, Anqi Ren, and Chetan Arora. PersonaGen: A tool for generating personas from user feedback. In *2023 IEEE 31st International Requirements Engineering Conference (RE)*, pages 353–354. IEEE, 2023.

Table 5: The natural language user queries on GoogleStoreApp dataset.

Question	Difficulty	Type	Comprehensiveness	Correctness	Readability
What topic has the most negative sentiment score on average?	Easy	Analysis	3.00	3.00	4.00
Create a word cloud for topics mentioned in Twitter posts in April .	Medium	Figure	5.00	4.33	5.00
Compare the sentiment of tweets mentioning 'WhatsApp' on weekdays versus weekends.	Hard	Analysis	4.67	3.67	4.67
Analyze the change in sentiment towards the 'Windows' product in April and May.	Medium	Analysis	4.67	3.67	4.67
What percentage of the total tweets in the dataset mention the product 'Windows'?	Easy	Analysis	4.00	3.67	4.33
Which topic appears most frequently in the Twitter dataset?	Easy	Analysis	4.33	4.67	4.67
What is the average sentiment score across all tweets?	Easy	Analysis	4.00	5.00	4.00
Determine the ratio of bug-related tweets to feature-request tweets for tweets related to 'Windows' product.	Medium	Analysis	4.33	4.67	4.67
Which top three timezones submitted the most number of tweets?	Easy	Analysis	4.67	4.67	5.00
Identify the top three topics with the fastest increase in mentions from April to May.	Medium	Analysis	3.33	4.33	4.00
In April, which pair of topics in the dataset co-occur the most frequently, and how many times do they appear together?	Medium	Analysis	4.67	4.67	5.00
Draw a histogram based on the different timezones, grouping timezones with fewer than 30 tweets under the category 'Others'.	Medium	Figure	4.67	5.00	5.00
What percentage of the tweets that mentioned 'Windows 10' were positive?	Easy	Analysis	4.67	5.00	4.67
How many tweets were posted in US during these months, and what percentage of these discuss the 'performance issue' topic?	Hard	Analysis	4.67	5.00	5.00
Check daily tweets occurrence on bug topic and do anomaly detection(Whether there was a surge on a given day).	Hard	Analysis	5.00	5.00	5.00
Which pair of topics in the dataset shows the highest statistical correlation in terms of their daily frequency of occurrence together during these months?	Medium	Analysis	4.67	4.33	4.67
Plot daily sentiment scores' trend for tweets mentioning 'Minecraft' in April and May.	Medium	Figure	4.67	5.00	5.00
Analyze the trend of weekly occurrence of topics 'bug' and 'performance issue'.	Medium	Figure	4.67	4.67	5.00
Analyze the correlation between the length of a tweet and its sentiment score.	Easy	Analysis	4.33	4.67	4.33
Which topics appeared in April but not in May talking about 'Instagram'?	Medium	Analysis	4.33	3.33	4.67
Identify the most common emojis used in tweets about 'CallofDuty' or 'Minecraft'.	Medium	Analysis	4.67	5.00	5.00
How many unique topics are there for tweets about 'Android'?	Easy	Analysis	4.00	5.00	4.67
What is the ratio of positive to negative emotions in the tweets related to the 'troubleshooting help' topic?	Medium	Analysis	4.67	5.00	4.67
Which product has highest average sentiment score?	Easy	Analysis	3.33	2.67	4.67
Plot a bar chart for the top 5 topics appearing in both April and May, using different colors for each month.	Hard	Figure	4.67	5.00	5.00
Find all the products related to game(e.g. Minecraft, CallofDuty) or game platform(e.g. Steam, Epic) yourself based on semantic information and knowledge. Then build a subset of tweets about those products. Get the top 5 topics in the subset and plot a pie chart.	Hard	Figure	4.00	3.67	4.33
Draw a issue river for the top 7 topics about 'WhatsApp' product.	Hard	Figure	4.67	4.33	4.33
Summarize 'Instagram' product advantages and disadvantages based on sentiment and tweets' content.	Hard	Suggestion	5.00	5.00	4.67
Based on the tweets, what action can be done to improve Android?	Hard	Suggestion	4.33	5.00	5.00
Based on the tweets in May, what improvements could enhance user satisfaction about Windows?	Hard	Suggestion	1.00	2.00	4.00

Table 6: The natural language user queries on ForumPost dataset.

Question	Difficulty	Type	Comprehensiveness	Correctness	Readability
What topic in the Forum Posts dataset has the highest average negative sentiment? If there are ties, list all possible answers.	Easy	Analysis	4.67	5.00	4.33
Create a word cloud for post content of the most frequently mentioned topic in Forum Posts.	Medium	Figure	4.33	5.00	4.67
Compare the sentiment of posts mentioning 'VLC' in different user levels.	Easy	Analysis	4.00	4.33	4.00
What topics are most often discussed in posts talking about 'user interface'?	Easy	Analysis	4.67	5.00	4.00
What percentage of the total forum posts mention the topic 'bug'?	Easy	Analysis	5.00	5.00	4.00
Draw a pie chart based on occurrence of different labels.	Easy	Figure	3.33	4.67	1.33
What is the average sentiment score across all forum posts?	Easy	Analysis	4.33	5.00	4.67
Determine the ratio of posts related to 'bug' to those related to 'feature request'.	Easy	Analysis	4.00	4.67	4.67
Which user level (e.g., new cone, big cone-huna) is most active in submitting posts?	Easy	Analysis	4.67	2.67	4.67
Order topic forum based on number of posts.	Easy	Analysis	4.33	5.00	4.67
Which pair of topics co-occur the most frequently, and how many times do they appear together?	Medium	Analysis	5.00	4.67	4.33
Draw a histogram for different user levels reflecting the occurrence of posts' content containing 'button'.	Medium	Figure	4.33	5.00	4.67
What percentage of posts labeled as application guidance are positive?	Easy	Analysis	4.33	5.00	4.67
How many posts were made by users at user level 'Cone Master'(case insensitive), and what percentage discuss 'installation issues'?	Medium	Analysis	4.67	5.00	4.67
Which pair of topics shows the highest statistical correlation in terms of their frequency of occurrence together?	Medium	Analysis	4.67	5.00	4.00
Plot a figure about the correlation between average sentiment score and different post positions.	Medium	Figure	4.00	4.00	3.67
Explore the correlation between the length of a post and its sentiment score.	Medium	Analysis	4.33	5.00	4.67
Which topics appeared frequently in posts with 'apparent bug' label?	Easy	Analysis	5.00	5.00	5.00
Identify the most common keywords used in posts about 'software configuration' topic.	Medium	Analysis	4.33	4.33	4.33
Identify the most frequently mentioned software or product names in the dataset.	Medium	Analysis	4.33	2.67	5.00
Draw a histogram about different labels for posts position is 'original post'.	Medium	Figure	4.00	4.67	4.00
What percentage of posts about 'UI/UX' is talking about the error of button.	Hard	Analysis	4.33	2.33	4.67
What is the biggest challenge faced by Firefox.	Hard	Analysis	2.00	3.00	4.00
What is the plugin mentioned the most in posts related to 'plugin issue' topic.	Medium	Analysis	3.67	2.33	4.67
What percentage of the posts contain url?	Medium	Analysis	3.33	3.00	4.67
Find the topic that appears the most and is present in all user levels, then draw a bar chart. Use different colors for different user-levels.	Medium	Figure	5.00	5.00	5.00
Based on the posts labeled as 'requesting more information', provide some suggestions on how to provide clear information to users.	Hard	Suggestion	5.00	4.33	5.00
Based on the most frequently mentioned issues, what improvements could be suggested for the most discussed software or hardware products?	Hard	Suggestion	3.33	4.00	4.00
Based on the posts with topic 'UI/UX', give suggestions on how to improve the UI design.	Hard	Suggestion	4.33	4.33	4.33
Based on the posts with 'application guidance' label, give suggestions on how to write better application guidance.	Hard	Suggestion	4.33	3.67	4.67

Table 7: The natural language user queries on MSearch dataset.

Question	Difficulty	Type	Comprehensiveness	Correctness	Readability
How many feedback are without query text?	Easy	Analysis	4.67	5.00	4.67
Which feedback topic have the most negative sentiment score on average?	Easy	Analysis	3.00	3.33	4.33
Which topics appeared in October but not in November?	Medium	Analysis	4.67	5.00	4.33
Plot a word cloud for translated feedback text with 'AI mistake' topic.	Easy	Figure	4.67	5.00	5.00
How many unique topics are there?	Easy	Analysis	4.67	5.00	5.00
What is the ratio of positive to negative emotions in the feedback related to 'others' topic?	Easy	Analysis	5.00	5.00	4.67
Which week are users most satisfied(highest average sentiment) with their search?	Hard	Analysis	5.00	5.00	4.33
Identify the top three topics with the fastest increase in occurrences from October to November.	Medium	Analysis	4.33	5.00	4.33
What are the top three topics in the dataset that have the lowest average sentiment scores?	Easy	Analysis	3.67	3.33	4.67
Plot a bar chart for top5 topics appear in both Oct and Nov. Oct use blue color and Nov's use orange color.	Hard	Figure	4.00	4.00	2.00
In October 2023, which pair of topics in the dataset co-occur the most frequently, and how many times do they appear together?	Hard	Analysis	3.00	3.33	4.33
Which pair of topics in the dataset shows the highest statistical correlation in terms of their daily frequency of occurrence together across the entire dataset?	Medium	Analysis	4.67	4.67	4.33
Find a subset that the feedback text contains information related to image. Get the top5 topics in the subset and plot a pie chart.	Hard	Figure	4.00	3.67	3.67
Draw an issue river for top 7 topics.	Hard	Figure	4.33	4.67	4.67
Plot a word cloud for topics in October 2023.	Medium	Figure	4.67	4.67	5.00
Identify the top three topics based on occurrence.	Easy	Analysis	5.00	5.00	5.00
Based on the data, what can be improved to the search engine given the most frequent topic?	Hard	Suggestion	5.00	4.67	4.00
Draw a histogram based on the different countries.	Medium	Figure	2.00	3.00	4.00
Plot daily sentiment scores' trend.	Medium	Figure	4.67	5.00	4.33
Draw a histogram based on the different countries. Group countries with fewer than 10 feedback entries under the category 'Others'.	Hard	Figure	4.00	4.00	4.00
Based on the data, what can be improved to improve the users' satisfaction?	Hard	Suggestion	4.67	4.67	4.33
What is the time range covered by the feedbacks?	Easy	Analysis	4.67	4.00	4.67
What percentage of the total queries in the dataset comes from US(country and region is us)	Easy	Analysis	5.00	5.00	5.00
Which topic appears most frequently?	Easy	Analysis	4.67	5.00	5.00
What is the average sentiment score across all feedback?	Easy	Analysis	4.67	5.00	4.33
How many feedback entries are labeled as 'unhelpful or irrelevant results' in topics?	Easy	Analysis	4.67	5.00	5.00
Which top three countries submitted the most number of feedback?	Easy	Analysis	5.00	5.00	5.00
Give me the trend of weekly occurrence of topic 'AI mistake' and 'AI image generation problem'	Medium	Figure	4.00	4.00	3.00
What percentage of the sentences that mentioned 'Bing AI' were positive?	Easy	Analysis	4.33	5.00	4.67
How many feedback entries submitted in German, and what percentage of these discuss 'slow performance' topic?	Hard	Analysis	3.67	1.00	4.67